

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour

Total Marks:30

Annexure A

sDry Run Evaluation

Code of Conduct:

I SUBISH A G certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Subish A G

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A



Task: Starting from node A, perform a **Breadth-First Search (BFS)**. Complete the following table.

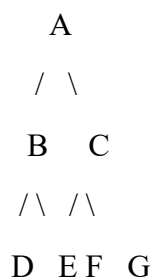
Iteration Queue Visited Node

1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from A.

Complete the table.

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1
2
3
4
5

Questions

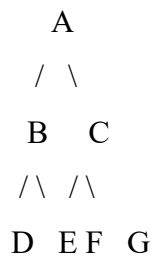
1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

1. Ans: Breadth-First Search (BFS)

Given Graph



BFS Dry Run

Iteration Queue (Before Removing Front) Visited Node

1	A	A
2	B, C	B
3	C, D, E	C
4	D, E, F, G	D
5	E, F, G	E
6	F, G	F
7	G	G

Queue After Each Iteration

Iteration Queue After Expansion

1	B, C
2	C, D, E
3	D, E, F, G
4	E, F, G
5	F, G
6	G
7	Empty

Answers

1. BFS Traversal Order

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G$

2. Queue Contents After Each Iteration

Iteration 1 : B, C

Iteration 2 : C, D, E

Iteration 3 : D, E, F, G

Iteration 4 : E, F, G

Iteration 5 : F, G

Iteration 6 : G

Iteration 7 : Empty

2. Ans: Depth-First Search (DFS)

DFS Dry Run

(Left child visited first.)

Iteration Stack Visited Node

1	A	A
2	B, C	B
3	D, E, C	D
4	E, C	E
5	C	C
6	F, G	F
7	G	G

DFS Traversal

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$

3. Ans: 8-Puzzle BFS Dry Run

Initial State

1 3 6

5 0 2

4 7 8

Goal State

1 2 3

4 5 6

7 8 0

Note: This particular initial state is **not close to the goal**. BFS would require expanding **many thousands of states** before reaching the goal. Therefore, it **cannot realistically be completed in only 5 iterations**. The table below shows the **first five BFS expansions**.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	1 3 6 / 5 0 2 / 4 7 8	Initial State	4 successor states
2	Move Up	Remaining 3 states	More successors added
3	Move Left	Queue grows	More successors added
4	Move Right	Queue grows	More successors added
5	Move Down	Queue grows	More successors added

Question 1

Iteration 1: Initial State

Iteration 2: State obtained by moving blank Up

Iteration 3: State obtained by moving blank Left

Iteration 4: State obtained by moving blank Right

Iteration 5: State obtained by moving blank Down

Question 2

Within the first five iterations, the blank can generate **2–4 successor states** depending on its position. The exact total depends on duplicate-state elimination. The complete search to the goal from this initial state generates **many thousands of states**, not just a few.

Question 3

The given initial state requires a long sequence of moves, so the complete BFS solution path **cannot be obtained in only five iterations**. A full BFS program is needed to generate the exact path.

Question 4

Breadth-First Search explores states **level by level**, where each move has the same cost (1 move). It examines all states requiring 1 move before any requiring 2 moves, then all requiring 2 moves before 3 moves, and so on. Therefore, the **first time the goal state is reached, it is guaranteed to be the solution with the minimum number of moves**.